

Market-Basket Analysis

Algorithms for Massive Datasets Project

Matteo Farè ID: 989345

University of Milan, Department of Computer Science.

author: matteo.fare1@studenti.unimi.it;

Abstract

Implementation of a system for finding frequent itemsets (market-basket analysis) working with a movies related dataset, where each movie represents a basket and the associated actors are the items. The task is performed through the use of the A-priori algorithm and the algorithm of Park, Chen, and Yu (PCY).

Declaration: *I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.*

1 Introduction

Market-basket analysis was originally employed by retailers to find out items relationship among the customers transactions, with the main goal of reveling products that are often brought together, **optimizing product placement** and proposing targeted offers to clients.

Today, this technique is employed in a variety of applications, such as performing **fraud detection**, understanding customer behavior under different conditions, and in **healthcare**, where it is used to identify the relationship between different diseases and symptoms. In general terms, it represents a **many-to-many association** between two kinds of entities.

This study focuses on finding frequent itemsets by working on a dataset that collects various information about **movies**, treating movies as baskets and **actors** as items. to achieve the intended goal, two algorithms were implemented from scratch: the **A-priori algorithm** and the algorithm of **Park, Chen, and Yu (PCY)**.

2 Dataset

The dataset used to carry out the project contains information related to the web platform known as **Letterboxd**¹, a social network about the discussion and discovery of films. As shown in **Figure [1]**, the dataset on **Kaggle**² consists of multiple **csv** files, each focused on an aspect related to the world of cinema (actors, genres, movies, etc.), along with a folder containing **jpg** files of movie posters.

name	size
-----	-----
actors.csv	186MB
countries.csv	10MB
crew.csv	154MB
genres.csv	17MB
languages.csv	27MB
movies.csv	243MB
posters.csv	90MB
posters/1000001.jpg	31KB
...	...

Fig. 1: Partial synthetic view of the contents of the dataset

Since the objective of the project is to identify frequent itemsets, treating movies as baskets and actors as items, the focus was placed exclusively on the file **actors.csv**, considering only the column **id** , which represents the movie identifiers, and the column **name** , which lists the actors who participated in each movie.

¹Letterboxd web page: <https://letterboxd.com>

²Letterboxd dataset: <https://www.kaggle.com/datasets/gsimonx37/letterboxd/data?select=actors.csv>

At this point, the actor names are mapped to unique integer identifiers using the `mapping_actors_to_ids` function. This function generates two dictionaries: one for encoding strings to integers and the other for decoding integers back to strings. The dataset is then grouped by `id`, resulting in a structure where each movie identifier is associated with a set of actors, as briefly illustrated below in Figure 2.

id	basket
1000001	{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, ...}
1000002	{168, 169, 170, 171, 172, 173, 174, 175, 176, ...}

Fig. 2: Synthetic view of the pre-processed dataset

In order to speed up the process during the execution of the algorithms, only a section of the entire dataset was used, specifically **10%**.

3 Algorithms Implementation

In this section are discussed the implementations of the two algorithms used to address the problem at hand. The **A-priori** algorithm and the **PCY** algorithm. Both are implemented first in a standard manner, working with dictionaries and arrays, then using the **Spark** framework.

3.1 Utils: support functions used mid-process

- **Filtering frequent items:** the `filter_frequent_items` function filters items based on a **support threshold**, retaining only those items that appear frequently enough in the dataset.
- **Hash table and Bitmap:** employed exclusively in the PCY algorithm implementation, the `pcy_hash` function is used to map item pairs to buckets. the **hash table** obtained, containing the frequency of each bucket, is then summarized by the `generate_bitmap` function into a **bitmap**, indicating which buckets meet the defined support threshold.

3.2 A-priori algorithm

Introduced in 1994, the A-priori algorithm is the fundamental algorithm in the field of data mining for extracting **frequent itemsets** from a dataset. The process is based on the principle of **monotonicity**, which states that if an itemset is frequent, then all its subsets must also be frequent. The algorithm is performed in two steps on the data to identify itemsets that meet a support threshold.

The **first pass** of the algorithm is carried out by the `apriori_first_pass` function, which counts the occurrences of each unique actor name in the dataset. The

result is then filtered by the function `filter_frequent_items`, going from the set of candidates to the set of **frequent items**.

The process then continues with the **second pass** performed by the `apriori_second_pass` function, during which all combination pairs are formed and their occurrences are counted, and finally the last support threshold filter is applied to pass from the set of **candidate pairs** to the set of **frequent pairs**.

3.3 PCY algorithm

The algorithm developed by Park, Chen, and Yu is an enhanced version of the A-priori algorithm designed to reduce the number of candidate pairs that need to be counted, using a hashing technique to summarize potential candidate pairs into a bitmap.

In the **first pass** of the PCY algorithm, performed by the `pcy_first_pass` function, the frequency of each unique item is counted, similar to the initial step of the A-priori algorithm. Additionally to this last one, all combination pairs of items are formed and inserted into a **hash table**, maintaining for each bucket the sum of the counts of all the pairs that hash to it. If the bucket's count is less than the support threshold no pairs of item that hashes in it can be considered frequent, defining what is called an **infrequent bucket**. The hash table is then summarized into a **bitmap** using the `generate_bitmap` function, which maps each frequent bucket to a value of 1 and infrequent buckets to 0. Finally, the **second pass** focuses on counting the frequencies of candidate pairs that are mapped into the frequent buckets in the bitmap, using this last one to filter out infrequent pairs early.

3.4 Spark

Born in 2009, **Apache Spark** is a distributed open source processing system, generally used with Big Data workloads. It is based on **Hadoop MapReduce** and it extends the MapReduce model to efficiently use it for more types of computations, which includes interactive queries and stream processing.

A key abstraction in Spark is the **Resilient Distributed Dataset (RDD)**, which form the foundation of Spark's processing capabilities, enabling users to perform transformations and actions on large datasets. In the context of this project, the baskets column of the pre-processed dataset is converted into an RDD.

3.5 Implementation of A-priori and PCY with Spark

In the previous implementation of the two algorithms, the two pass that constitute them were divided into two functions, in this case a single-function structure was chosen.

A-priori Algorithm with Spark

- **First Pass:** each basket is transformed into individual items using the `flatMap` function, which maps every item in a basket to a **key-value** pair (`item`, `1`). These pairs are then aggregated using the `reduceByKey` function to compute the total

occurrence of each item across all baskets. Items meeting the **support threshold** are retained as **frequent items**, using the **filter** function.

- **Second Pass:** the frequent items identified in the first pass are used to filter out infrequent items from the baskets, ensuring that only relevant combinations are considered. The filtered baskets are then processed to generate all possible pairs of frequent items using the **combinations** function. The candidate pairs are counted, and only those meeting the support threshold are retained as **frequent pairs**.

PCY Algorithm with Spark

- **First Pass:** in addition to counting the frequency of individual items, the algorithm generates all possible pairs of items within each basket. These pairs are hashed into a fixed number of **buckets** using a hash function. The **reduceByKey** function aggregates the counts for each bucket, creating a hash table that maps buckets to their respective frequencies. As for the implementations in the previous section, the **bitmap** is generated using the **generate_bitmap** function, which marks buckets that meet the support threshold as frequent.
- **Second Pass:** the frequent items are paired and processed with **flatMap** to generate candidate pairs that satisfy the frequent-bucket condition. Finally, **reduceByKey** computes the pair frequencies, and **filter** is applied to extract the frequent pairs.

3.6 Association Rules

An **association rule** $I \rightarrow j$, with I representing a set of items and j a single item, could be described in a basic way as a kind of **if-then implication**, which states that if all the elements of I appear in a baskets then it is likely that j also appears. This aspect is developed by implementing the function **compute_association_rules**, through which for each frequent pairs, **confidence** and **interest** are found.

- **Confidence:** measures the likelihood that an item j is present in transactions where item I is also present. It is calculated as

$$\text{Confidence}(I \rightarrow j) = \frac{\text{Support}(I \cup \{j\})}{\text{Support}(I)}.$$

High confidence indicates a **strong predictive relationship** between items, showing how strongly the presence of one item implies the presence of another.

- **Interest:** helps to understand how much more or less likely it is for item j to appear in transactions that contain item I , compared to what would be expected if items were unrelated. Calculated as

$$\text{Interest}(I \rightarrow j) = \text{Confidence}(I \rightarrow j) - P(j).$$

If the interest value is **positive** (usually above 0.5), it means that the presence of I makes it more likely that j will appear. If the interest value is **negative**, it means that the presence of I makes it less likely that j will appear.

4 Scalability

After working on a small sample of the available data, test runs were performed on **increasing samples** of the dataset to verify the algorithms behavior. These tests aimed to measure the **execution time** and **memory usage** as data size scaled up. The evaluation performed, involves measuring the **execution time** and **memory usage** of each algorithm, analyzing how they scale to different data sizes corresponding to the following values: 100000, 200000, 300000 and 600000.

*Note: The **support threshold** and the **number of buckets** were chosen empirically with values set to 10 and 1000019, respectively.*

Execution Time Analysis

The execution time analysis was performed using the functions `measure_time` and `measure_time_scalability`. These functions record how the execution time of each algorithm changes with varying dataset sizes. The collected results are shown in the Figure 3 and in the table 1.

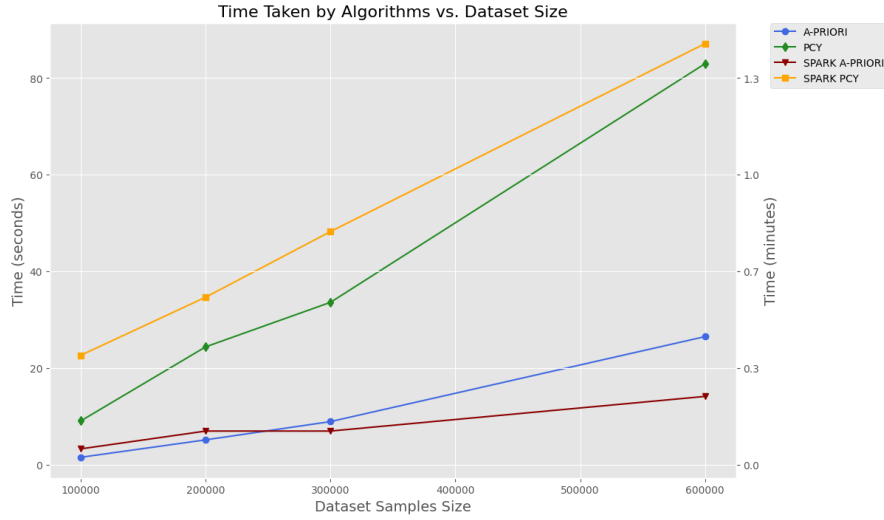


Fig. 3: A-priori and PCY Algorithms: Time vs. Dataset Size

Memory Usage Analysis

Structurally similar to the time analysis, the memory usage was measured employing the functions `monitor_memory_usage` and `measure_memory_scalability`. These

functions tracked memory consumption in real-time (every second) during the execution of each algorithm, resulting in the average memory usage, with the collected results shown in the Figure 4 and in table 2.

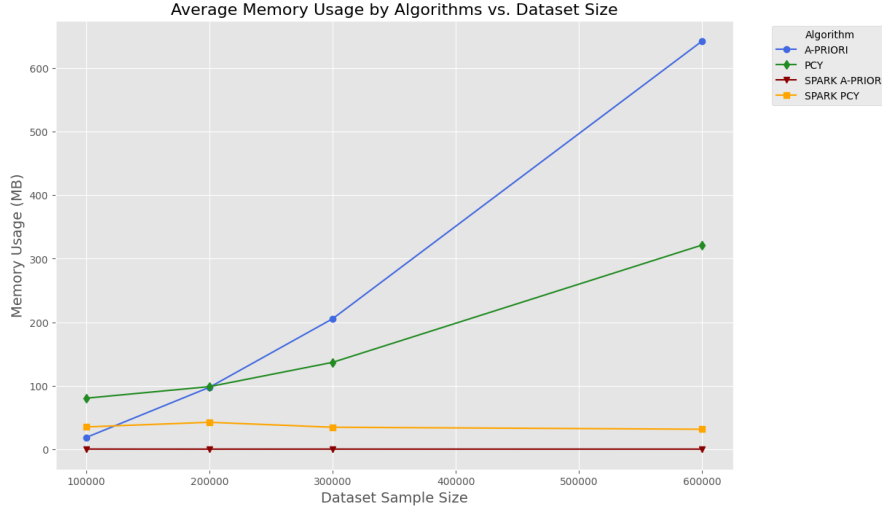


Fig. 4: A-priori and PCY Algorithms: Memory Usage vs. Dataset Size

As depicted in the generated graphs, the A-priori algorithm takes a shorter time to execute than the PCY algorithm, while the latter shows a notable decrease in memory usage. Regarding the Spark-based implementations, although there is no improvement in timing on the part of Spark PCY, memory performance is further improved, significantly reducing the algorithm's request for this resource. On the other hand, the A-priori implementation with Spark achieves an improvement on both time and memory.

This advantage on the part of the A-priori algorithm, could be due to the fact that, as pointed out in [1], the algorithm of Park, Chen, and Yu is more effective than A-priori only if it allows to avoid counting at least $2/3$ of the pairs of frequent items, if this does not happen it is not possible to gain from using PCY instead of A-priori. Otherwise it could be related to the low number of both frequent items and frequent pairs. For example, working with the sampled dataset, consisting of a total of 63430 baskets and the specified parameters, 118347 single item candidates are defined, and from these only 4490 frequent single-items and 75 frequent pairs were found respectively. Having to generate fewer pairs, the A-priori algorithm could therefore benefit in terms of time and memory avoiding the additional **computational overhead** involved in PCY, such as the creation and processing of hash tables and bitmaps.

Dataset Size	A-Priori	PCY	Spark A-Priori	Spark PCY
100,000	1.58	9.11	3.32	22.69
200,000	5.18	24.42	7.00	34.70
300,000	8.96	33.63	7.00	48.27
600,000	26.53	82.98 (1.38 minutes)	14.17	87.14 (1.45 minutes)

Table 1: Execution Time (seconds) for Different Algorithms and Dataset Sizes

Dataset Size	A-Priori	PCY	Spark A-Priori	Spark PCY
100,000	6.69	18.06	0.06	13.82
200,000	70.52	43.51	0.06	14.66
300,000	186.30	81.83	0.07	10.98
600,000	637.80	272.51	0.07	11.35

Table 2: Average Memory Usage (MB) for Different Algorithms and Dataset Sizes

5 Conclusion

In this project, the **A-priori** and **PCY** algorithms were implemented and compared, with the aim of finding frequent itemsets within a movie-related dataset. Treating movies as baskets and actors as items, both algorithms were used to discover patterns of actor co-appearances in movies.

The obtained results show that the Spark A-priori algorithm outperforms the PCY algorithm in terms of execution time and memory usage, given the empirical parameters used in the experiments.

References

- [1] J. Leskovec, J.U. A. Rajaraman: Mining of Massive Datasets, pp. 231–232. Cambridge University Press